

Work description and log

Data collection and storage

For both production and consumption I retrieve **hourly time series for all Norwegian price areas (NO1–NO5)** from the Elhub API:

- I use separate datasets for production and consumption, such as `PRODUCTION_PER_GROUP_MBA_HOUR` and `CONSUMPTION_PER_GROUP_MBA_HOUR`.
- A helper function requests one time window, one area and one dataset at a time, respecting the API's date limits.
- I loop over all price areas and all relevant years (2021–2024), then combine the results into Spark DataFrames.

The storage strategy is:

- **Cassandra**
 - I store hourly production data (including the new years 2022–2024) in Cassandra, extending the solution from part 2 of the project.
 - I also store hourly consumption data for 2021–2024 in Cassandra in a similar way.
 - Cassandra is the “full archive” for both production and consumption time series.
- **MongoDB**
 - Hourly production data for 2022–2024 (and the curated 2021 data) are also written to MongoDB, so that the Streamlit app can query pre-processed production data directly.
 - For consumption (2021–2024), the code to send data to MongoDB is implemented and tested, but when I tried to upload the full dataset to MongoDB Atlas, I hit the free-tier **space quota error (517 MB of 512 MB)**.
 - This limitation is due to the free MongoDB Atlas account, not the logic of the solution. Because of this, full consumption storage is only guaranteed in Cassandra, while MongoDB is mainly used for production (and for smaller or derived subsets of consumption when needed for the app).

In the notebook I also include an **overview table** that clearly summarizes which datasets ended up in Cassandra and MongoDB:

What was difficult

Several parts of this assignment were technically challenging:

- **Environment and dependencies**

- Getting Spark, Python and the relevant connectors (Cassandra and MongoDB) to work together reliably was time-consuming.
- I had to deal with version mismatches, SSL/certificate issues and connection errors before the pipeline ran smoothly.
- **API and data volume**
 - The Elhub API has time window limitations, so I had to design loops that break the period into smaller ranges and still bring everything back together consistently.
 - The size of the consumption dataset made it problematic to store everything in MongoDB Atlas, leading to the free-tier storage quota error.
- **Databases and verification**
 - I spent time checking that the data in Cassandra and MongoDB actually matched expectations (row counts, date ranges, groups, and price areas), and debugging when they did not. Even though these issues were frustrating at times, they forced me to understand the tools (Spark, Cassandra, MongoDB, the Elhub API) much better.

What was enjoyable

There were also several parts of the work that I found genuinely fun and rewarding:

- **End-to-end pipeline**
 - It was satisfying to see the full pipeline working: from raw Elhub API responses, through Spark transformations, into Cassandra/MongoDB, and finally being available for the Streamlit app and visualizations.
 - Having both production and consumption data aligned by hour and price area made it feel like a "real" data engineering task.
- **Exploration and structure**
 - Building and documenting the data collection and storage steps in a structured way made it easier to reason about the rest of the project (mapping, correlations, forecasting, etc.).
 - It was also nice to summarize everything in the overview table so that the final state of the databases is clear.
- **Problem-solving**
 - Even though the MongoDB quota issue was annoying, it was also a good lesson in practical limitations of cloud services.
 - Solving the connection issues and finally seeing the data appear correctly in both Cassandra and MongoDB was a good feeling.

Overall, the notebook shows how I approached the assignment as a complete data pipeline task: plan the data flow, implement it step by step, handle real-world obstacles (APIs, connectors, quotas), and document what actually works in the final solution.

Repository and Streamlit App

- Student: Abdul Haadi Sheikh
- GitHub repository:
[IND320_1_Project – branch utvikling_v4](#)
- Streamlit app:
<https://streamv4.streamlit.app/>

AI usage

During this assignment I have used AI tools only for **debugging** and for **learning how to use new commands and workflows** (for example: understanding error messages, trying out `git` /terminal commands, configuring MongoDB/Spark/Streamlit, and clarifying how certain functions or libraries work).

All core code, data processing logic, and implementation decisions in the notebook and Streamlit app have been written and structured by myself. I have not used AI to generate or complete substantial parts of the actual solution code.

In addition to AI-based debugging help, I have also received very good help from the teaching assistant and from Kristian on Teams, especially when I was stuck on technical issues or needed clarification of the assignment requirements. I am grateful for their support and explanations.

```
In [14]: import requests
import pandas as pd
from datetime import datetime
from dateutil.relativedelta import relativedelta

from pyspark.sql.types import StructType, StructField, StringType, DoubleType
print(" Imports for Elhub is ready")
```

Imports for Elhub is ready

Elhub data pipeline: Spark → Cassandra and MongoDB (2021–2024)

```
In [15]: import os
import sys
import subprocess

# Stop old SparkSession if it exists
try:
    spark.stop()
except:
    pass

# --- Set up JAVA_HOME (try to find Java 11 automatically on macOS) ---
try:
```

```

    java_home = subprocess.check_output(
        ["/usr/libexec/java_home", "-v", "11"]
    ).decode().strip()
    os.environ["JAVA_HOME"] = java_home
    print("JAVA_HOME set to:", java_home)
except Exception as e:
    print("Could not find Java 11 with /usr/libexec/java_home.")
    print("    Check that you have Java 11 installed. Error:", e)

# Make sure PySpark uses the same Python as Jupyter
os.environ["PYSPARK_PYTHON"] = sys.executable
os.environ["PYSPARK_DRIVER_PYTHON"] = sys.executable

from pyspark.sql import SparkSession

# --- SparkSession with Cassandra + correct driver address ---
spark = (
    SparkSession.builder
        .appName("Elhub Production & Consumption 2021-2024")
        .master("local[*]") # local execution
        # These two are important for the error you saw
        .config("spark.driver.bindAddress", "127.0.0.1")
        .config("spark.driver.host", "127.0.0.1")
        # Cassandra connector
        .config(
            "spark.jars.packages",
            "com.datastax.spark:spark-cassandra-connector_2.12:3.5.0"
        )
        .config("spark.cassandra.connection.host", "127.0.0.1")
        .config("spark.cassandra.connection.port", "9042")
        .config("spark.cassandra.auth.username", "cassandra")
        .config("spark.cassandra.auth.password", "cassandra")
        .config("spark.cassandra.output.consistency.level", "LOCAL_QUORUM")
        .config("spark.sql.extensions", "com.datastax.spark.connector.Cassand
        .getOrCreate()
)

print("SparkSession connected to Cassandra (driver at 127.0.0.1)")

# --- MongoDB client via secrets.toml ---
import tomllib # built into Python 3.11+
import certifi
from pymongo.mongo_client import MongoClient
from pymongo.server_api import ServerApi

# Read .streamlit/secrets.toml
with open("/Users/a.h.sheikh/Desktop/IND320_Git_Job/IND320_1_Project/.str
    secrets = tomllib.load(f)

MONGODB_URI = secrets["MONGODB_URI"] # same as in the Streamlit app

ca = certifi.where()
client = MongoClient(MONGODB_URI, server_api=ServerApi("1"), tlsCAFile=ca

# Test connection
try:
    client.admin.command("ping")
    print(" MongoDB connection OK")
except Exception as e:

```

```

print(" MongoDB connection failed:", e)

# Choose database and collections (same as before, or rename if you want)
mongo_db = client["example"]
mongo_collection_production = mongo_db["production"] # production 2021
mongo_collection_consumption = mongo_db["consumption"] # consumption 202

# WARNING: this deletes the entire database "example"
client.drop_database("example")
print("🔪 Dropped database 'example'")

# %% [code]
# Elhub API: common setup and helper functions
BASE_URL = "https://api.elhub.no/energy-data/v0/price-areas"

# Fetch price areas dynamically (fallback N01-N05 if something fails)
areas_response = requests.get(BASE_URL)
if areas_response.status_code == 200:
    areas = areas_response.json().get("priceAreas", ["N01", "N02", "N03",
else:
    print(" Could not fetch priceAreas from API. Using N01-N05 as default
    areas = ["N01", "N02", "N03", "N04", "N05"]

print(" Price areas:", areas)

def fetch_elhub_window(dataset: str,
                        record_key: str,
                        area: str,
                        start_dt: datetime,
                        end_dt: datetime) -> pd.DataFrame:
    """
    Fetch one time window for one area and one dataset.

    Args:
        dataset: e.g. 'PRODUCTION_PER_GROUP_MBA_HOUR'
        record_key: e.g. 'productionPerGroupMbaHour' / 'consumptionPerGro
        area: price area, e.g. 'N01'
        start_dt: start datetime (with timezone)
        end_dt: end datetime (with timezone)

    Returns:
        A pandas DataFrame containing the records for the given window an
        Returns an empty DataFrame if something fails or no data is retur
    """
    api_url = f"{BASE_URL}/{area}"
    params = {
        "dataset": dataset,
        "startDate": start_dt.isoformat(), # requests URL-encodes + to %
        "endDate": end_dt.isoformat()
    }

    response = requests.get(api_url, params=params)

    if response.status_code != 200:
        print(
            f" Error {response.status_code} for {dataset}, {area}: "
            f"{start_dt.isoformat()} -> {end_dt.isoformat()}"
        )
    return pd.DataFrame()

```

```

try:
    data = response.json()
    records = data["data"][0]["attributes"][record_key]
except (KeyError, IndexError):
    print(
        f" Unexpected JSON structure for {dataset}, {area}: "
        f"{start_dt.isoformat()} → {end_dt.isoformat()}"
    )
    return pd.DataFrame()

if not records:
    print(
        f" No rows for {dataset}, {area}: "
        f"{start_dt.isoformat()} → {end_dt.isoformat()}"
    )
    return pd.DataFrame()

df = pd.DataFrame(records)
df["priceArea"] = area
return df

def fetch_dataset_years(dataset: str,
                        record_key: str,
                        years,
                        areas) -> pd.DataFrame:
    """
    Fetch data for several years and all price areas, using monthly window
    (to stay within the maximum date range allowed by the API).

    Args:
        dataset: dataset name, e.g. 'PRODUCTION_PER_GROUP_MBA_HOUR'
        record_key: key inside the JSON, e.g. 'productionPerGroupMbaHour'
        years: iterable of years (e.g. range(2022, 2025))
        areas: list of price areas (e.g. ['N01', ..., 'N05'])

    Returns:
        A pandas DataFrame with concatenated data from all requested years
        Returns an empty DataFrame if no data is fetched.
    """
    all_dfs = []

    for year in years:
        year_start = datetime.fromisoformat(f"{year}-01-01T00:00:00+02:00")
        year_end = datetime.fromisoformat(f"{year}-12-31T23:59:59+02:00")
        print(f"\n📅 Fetching {dataset} for year {year}")

        for area in areas:
            current_start = year_start
            while current_start < year_end:
                current_end = current_start + relativedelta(months=1)
                if current_end > year_end:
                    current_end = year_end

                print(
                    f"    → {dataset} {area}: "
                    f"{current_start.isoformat()} → {current_end.isoformat()}"
                )

```

```

        df = fetch_elhub_window(dataset, record_key, area, current_start, current_end)
        if not df.empty:
            all_dfs.append(df)

        current_start = current_end

    if not all_dfs:
        print("⚠ No data fetched.")
        return pd.DataFrame()

    final_df = pd.concat(all_dfs, ignore_index=True)
    return final_df

# ## Production 2022–2024 (PRODUCTION_PER_GROUP_MBA_HOUR)
# We assume that production data for 2021 already exists in the Cassandra
# Here we fetch 2022–2024 and **append** to the same table and to MongoDB

# %% [code]
# Fetch production data for 2022–2024
production_years = range(2022, 2025) # 2022, 2023, 2024

production_df_22_24 = fetch_dataset_years(
    dataset="PRODUCTION_PER_GROUP_MBA_HOUR",
    record_key="productionPerGroupMbaHour",
    years=production_years,
    areas=areas
)

print("\n Number of production rows 2022–2024:", len(production_df_22_24))

# Save to CSV for backup/check
production_df_22_24.to_csv("elhub_production_2022_2024_all_areas.csv", index=False)
print(" Saved to elhub_production_2022_2024_all_areas.csv")

# Write production 2022–2024 to Cassandra (append to existing table with
# Write production 2022–2024 to Cassandra (append to existing table with

# Select only the relevant columns for Cassandra
prod_columns = [
    "endTime",
    "lastUpdatedTime",
    "priceArea",
    "productionGroup",
    "quantityKwh",
    "startTime",
]

prod_for_spark = production_df_22_24[prod_columns].copy()

from pyspark.sql.types import StructType, StructField, StringType, DoubleType

prod_schema = StructType([
    StructField("endTime", StringType(), True),
    StructField("lastUpdatedTime", StringType(), True),
    StructField("priceArea", StringType(), True),
    StructField("productionGroup", StringType(), True),
    StructField("quantityKwh", DoubleType(), True),
    StructField("startTime", StringType(), True),
])

```

```

spark_prod_df = spark.createDataFrame(prod_for_spark, schema=prod_schema)

# First: use the same casing as we want in Cassandra
spark_prod_df = (
    spark_prod_df
    .withColumnRenamed("endTime", "endtime")
    .withColumnRenamed("lastUpdatedTime", "lastupdatedtime")
    .withColumnRenamed("priceArea", "pricearea")
    .withColumnRenamed("productionGroup", "productiongroup")
    .withColumnRenamed("quantityKwh", "quantitykwh")
    .withColumnRenamed("startTime", "starttime")
)

# IMPORTANT: select only the columns that actually exist in the table pr
spark_prod_df = spark_prod_df.select(
    "pricearea",
    "productiongroup",
    "starttime",
    "quantitykwh"
)

print("Schema sent to Cassandra:")
spark_prod_df.printSchema()

(
    spark_prod_df
    .write
    .format("org.apache.spark.sql.cassandra")
    .mode("append")
    .options(keyspace="elhub", table="production_2021")
    .save()
)

print(" Production 2022–2024 written to Cassandra (elhub.production_2021)")

# Write production 2022–2024 to MongoDB (append after 2021 data)
# Here we only include a few important columns
prod_for_mongo = production_df_22_24[[
    "priceArea",
    "productionGroup",
    "startTime",
    "quantityKwh"
]].copy()

prod_for_mongo.rename(
    columns={
        "priceArea": "pricearea",
        "productionGroup": "productiongroup",
        "startTime": "starttime",
        "quantityKwh": "quantitykwh",
    },
    inplace=True,
)

prod_docs = prod_for_mongo.to_dict("records")

try:
    if prod_docs:
        mongo_collection_production.insert_many(prod_docs)
        print(f" {len(prod_docs)} production rows 2022–2024 written to Mo

```



```

    else:
        print(" No production documents to write to MongoDB")
except Exception as e:
    print(" Could not write production to MongoDB:", e)

# ### Consumption 2021–2024 (CONSUMPTION_PER_GROUP_MBA_HOUR)
# Here we use **new tables** in Cassandra and MongoDB:
# – Cassandra: `elhub.consumption_2021_2024`
# – MongoDB: collection `consumption` in database `example`
#
# You must ensure that the Cassandra table exists, e.g. something like t
#
# ```sql
# CREATE TABLE elhub.consumption_2021_2024 (
#     pricearea text,
#     consumptiongroup text,
#     starttime text,
#     endtime text,
#     lastupdatedtime text,
#     quantitykwh double,
#     PRIMARY KEY ((pricearea, consumptiongroup), starttime)
# );
# ```

# %% [code]
# Fetch consumption data 2021–2024
consumption_years = range(2021, 2025) # 2021, 2022, 2023, 2024

consumption_df_21_24 = fetch_dataset_years(
    dataset="CONSUMPTION_PER_GROUP_MBA_HOUR",
    record_key="consumptionPerGroupMbaHour",
    years=consumption_years,
    areas=areas
)

print("\n Number of consumption rows 2021–2024:", len(consumption_df_21_24))

# Save to CSV for inspection
consumption_df_21_24.to_csv("elhub_consumption_2021_2024_all_areas.csv",
print(" Saved to elhub_consumption_2021_2024_all_areas.csv")

# %% [code]
# Write consumption 2021–2024 to Cassandra (NEW table)

cons_columns = [
    "endTime",
    "lastUpdatedTime",
    "priceArea",
    "consumptionGroup",
    "quantityKwh",
    "startTime",
]

cons_for_spark = consumption_df_21_24[cons_columns].copy()

cons_schema = StructType([
    StructField("endTime", StringType(), True),
    StructField("lastUpdatedTime", StringType(), True),
    StructField("priceArea", StringType(), True),
    StructField("consumptionGroup", StringType(), True),
    StructField("quantityKwh", DoubleType(), True),

```

```

    StructField("startTime", StringType(), True),
])

spark_cons_df = spark.createDataFrame(cons_for_spark, schema=cons_schema)

spark_cons_df = (
    spark_cons_df
    .withColumnRenamed("endTime", "endtime")
    .withColumnRenamed("lastUpdatedTime", "lastupdatedtime")
    .withColumnRenamed("priceArea", "pricearea")
    .withColumnRenamed("consumptionGroup", "consumptiongroup")
    .withColumnRenamed("quantityKwh", "quantitykwh")
    .withColumnRenamed("startTime", "starttime")
)

(
    spark_cons_df
    .write
    .format("org.apache.spark.sql.cassandra")
    .mode("append")
    .options(keyspace="elhub", table="consumption_2021_2024") # NEW table
    .save()
)

print(" Consumption 2021-2024 written to Cassandra (elhub.consumption_2021_2024)")

# Write consumption 2021-2024 to MongoDB (NEW collection `consumption`)

cons_for_mongo = consumption_df_21_24[[
    "priceArea",
    "consumptionGroup",
    "startTime",
    "quantityKwh"
]].copy()

cons_for_mongo.rename(
    columns={
        "priceArea": "pricearea",
        "consumptionGroup": "consumptiongroup",
        "startTime": "starttime",
        "quantityKwh": "quantitykwh",
    },
    inplace=True,
)

cons_docs = cons_for_mongo.to_dict("records")

try:
    if cons_docs:
        mongo_collection_consumption.insert_many(cons_docs)
        print(f" {len(cons_docs)} consumption rows 2021-2024 written to MongoDB")
    else:
        print(" No consumption documents to write to MongoDB")
except Exception as e:
    print(" Could not write consumption to MongoDB:", e)

```

JAVA_HOME set to: /Library/Java/JavaVirtualMachines/zulu-11.jdk/Contents/Home

SparkSession connected to Cassandra (driver at 127.0.0.1)

MongoDB connection OK

🔧 Dropped database 'example'

Price areas: ['N01', 'N02', 'N03', 'N04', 'N05']

📄 Fetching PRODUCTION_PER_GROUP_MBA_HOUR for year 2022

→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-01-01T00:00:00+02:00 → 2022-02-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-02-01T00:00:00+02:00 → 2022-03-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-03-01T00:00:00+02:00 → 2022-04-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-04-01T00:00:00+02:00 → 2022-05-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-05-01T00:00:00+02:00 → 2022-06-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-06-01T00:00:00+02:00 → 2022-07-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-07-01T00:00:00+02:00 → 2022-08-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-08-01T00:00:00+02:00 → 2022-09-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-09-01T00:00:00+02:00 → 2022-10-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-10-01T00:00:00+02:00 → 2022-11-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-11-01T00:00:00+02:00 → 2022-12-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N01: 2022-12-01T00:00:00+02:00 → 2022-12-31T23:59:59+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-01-01T00:00:00+02:00 → 2022-02-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-02-01T00:00:00+02:00 → 2022-03-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-03-01T00:00:00+02:00 → 2022-04-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-04-01T00:00:00+02:00 → 2022-05-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-05-01T00:00:00+02:00 → 2022-06-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-06-01T00:00:00+02:00 → 2022-07-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-07-01T00:00:00+02:00 → 2022-08-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-08-01T00:00:00+02:00 → 2022-09-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-09-01T00:00:00+02:00 → 2022-10-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-10-01T00:00:00+02:00 → 2022-11-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-11-01T00:00:00+02:00 → 2022-12-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N02: 2022-12-01T00:00:00+02:00 → 2022-12-31T23:59:59+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N03: 2022-01-01T00:00:00+02:00 → 2022-02-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N03: 2022-02-01T00:00:00+02:00 → 2022-03-01T00:00:00+02:00

[illegible]

[illegible]

[illegible]


```
→ PRODUCTION_PER_GROUP_MBA_HOUR N05: 2024-07-01T00:00:00+02:00 → 2024-08-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N05: 2024-08-01T00:00:00+02:00 → 2024-09-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N05: 2024-09-01T00:00:00+02:00 → 2024-10-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N05: 2024-10-01T00:00:00+02:00 → 2024-11-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N05: 2024-11-01T00:00:00+02:00 → 2024-12-01T00:00:00+02:00
→ PRODUCTION_PER_GROUP_MBA_HOUR N05: 2024-12-01T00:00:00+02:00 → 2024-12-31T23:59:59+02:00
```

Number of production rows 2022-2024: 657600

Saved to elhub_production_2022_2024_all_areas.csv

Schema sent to Cassandra:

root

```
|-- pricearea: string (nullable = true)
|-- productiongroup: string (nullable = true)
|-- starttime: string (nullable = true)
|-- quantitykwh: double (nullable = true)
```

25/11/23 11:41:30 WARN TaskSetManager: Stage 0 contains a task of very large size (8510 KiB). The maximum recommended task size is 1000 KiB.

Production 2022–2024 written to Cassandra (elhub.production_2021)
657600 production rows 2022–2024 written to MongoDB

 Fetching CONSUMPTION_PER_GROUP_MBA_HOUR for year 2021

- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-01-01T00:00:00+02:00 → 2021-02-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-02-01T00:00:00+02:00 → 2021-03-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-03-01T00:00:00+02:00 → 2021-04-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-04-01T00:00:00+02:00 → 2021-05-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-05-01T00:00:00+02:00 → 2021-06-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-06-01T00:00:00+02:00 → 2021-07-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-07-01T00:00:00+02:00 → 2021-08-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-08-01T00:00:00+02:00 → 2021-09-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-09-01T00:00:00+02:00 → 2021-10-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-10-01T00:00:00+02:00 → 2021-11-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-11-01T00:00:00+02:00 → 2021-12-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N01: 2021-12-01T00:00:00+02:00 → 2021-12-31T23:59:59+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-01-01T00:00:00+02:00 → 2021-02-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-02-01T00:00:00+02:00 → 2021-03-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-03-01T00:00:00+02:00 → 2021-04-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-04-01T00:00:00+02:00 → 2021-05-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-05-01T00:00:00+02:00 → 2021-06-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-06-01T00:00:00+02:00 → 2021-07-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-07-01T00:00:00+02:00 → 2021-08-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-08-01T00:00:00+02:00 → 2021-09-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-09-01T00:00:00+02:00 → 2021-10-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-10-01T00:00:00+02:00 → 2021-11-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-11-01T00:00:00+02:00 → 2021-12-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N02: 2021-12-01T00:00:00+02:00 → 2021-12-31T23:59:59+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N03: 2021-01-01T00:00:00+02:00 → 2021-02-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N03: 2021-02-01T00:00:00+02:00 → 2021-03-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N03: 2021-03-01T00:00:00+02:00 → 2021-04-01T00:00:00+02:00
- CONSUMPTION_PER_GROUP_MBA_HOUR N03: 2021-04-01T00:00:00+02:00 → 2021-05-01T00:00:00+02:00

[illegible]

[illegible]

[illegible]

[illegible]

→ CONSUMPTION_PER_GROUP_MBA_HOUR N05: 2024-08-01T00:00:00+02:00 → 2024-09-01T00:00:00+02:00
 → CONSUMPTION_PER_GROUP_MBA_HOUR N05: 2024-09-01T00:00:00+02:00 → 2024-10-01T00:00:00+02:00
 → CONSUMPTION_PER_GROUP_MBA_HOUR N05: 2024-10-01T00:00:00+02:00 → 2024-11-01T00:00:00+02:00
 → CONSUMPTION_PER_GROUP_MBA_HOUR N05: 2024-11-01T00:00:00+02:00 → 2024-12-01T00:00:00+02:00
 → CONSUMPTION_PER_GROUP_MBA_HOUR N05: 2024-12-01T00:00:00+02:00 → 2024-12-31T23:59:59+02:00

Number of consumption rows 2021–2024: 876600

Saved to elhub_consumption_2021_2024_all_areas.csv

25/11/23 11:46:06 WARN TaskSetManager: Stage 1 contains a task of very large size (11637 KiB). The maximum recommended task size is 1000 KiB.

Consumption 2021–2024 written to Cassandra (elhub.consumption_2021_2024)
 876600 consumption rows 2021–2024 written to MongoDB

Loading and sanity-checking production_2021 from Cassandra

```
In [16]: from pyspark.sql.functions import substring, col, countDistinct

# Load production data for 2021 from Cassandra
# Keyspace: "elhub", table: "production_2021"
prod_cass = (
    spark.read
    .format("org.apache.spark.sql.cassandra")
    .options(keyspace="elhub", table="production_2021")
    .load()
)

# Quick sanity check: total number of rows in the Cassandra table
print("Total number of rows in elhub.production_2021:", prod_cass.count())

# Inspect a few sample rows (columns that are relevant for this assignment)
prod_cass.select("pricearea", "productiongroup", "starttime", "quantitykw")

# Add a "year" column by extracting the first 4 characters from starttime
prod_with_year = prod_cass.withColumn("year", substring(col("starttime"),

# Rough check: count rows per (year, pricearea) to see which years and areas
prod_with_year.groupBy("year", "pricearea").count().orderBy("year", "pricearea"))
```

Total number of rows in elhub.production_2021: 657601

pricearea	productiongroup	starttime	quantitykwh
N04	hydro	2022-01-01 00:00:00	3840983.2
N04	hydro	2022-01-01 01:00:00	3816076.8
N04	hydro	2022-01-01 02:00:00	3827141.0
N04	hydro	2022-01-01 03:00:00	3813465.8
N04	hydro	2022-01-01 04:00:00	3763152.5
N04	hydro	2022-01-01 05:00:00	3766277.5
N04	hydro	2022-01-01 06:00:00	3743969.8
N04	hydro	2022-01-01 07:00:00	3802487.5
N04	hydro	2022-01-01 08:00:00	3913795.2
N04	hydro	2022-01-01 09:00:00	3969193.2

only showing top 10 rows

[Stage 6:=====> (15 + 2)
/ 17]

year	pricearea	count
2021	N01	1
2022	N01	43800
2022	N02	43800
2022	N03	43800
2022	N04	43800
2022	N05	43800
2023	N01	43800
2023	N02	43800
2023	N03	43800
2023	N04	43800
2023	N05	43800
2024	N01	43920
2024	N02	43920
2024	N03	43920
2024	N04	43920
2024	N05	43920

Preparing and inserting 2022–2024 production data into MongoDB

```
In [17]: # Assumes that production_df_22_24 is still in memory
prod_for_mongo = production_df_22_24[[
    "priceArea",
    "productionGroup",
    "startTime",
    "quantityKwh"
]].copy()

prod_for_mongo.rename(
    columns={
        "priceArea": "pricearea",
        "productionGroup": "productiongroup",
        "startTime": "starttime",
        "quantityKwh": "quantitykwh",
    },
```

```

        inplace=True,
    )

    prod_docs = prod_for_mongo.to_dict("records")

    if prod_docs:
        mongo_collection_production.insert_many(prod_docs)
        print(f" {len(prod_docs)} production documents 2022–2024 written to MongoDB")
    else:
        print("No production documents to write to MongoDB")

```

657600 production documents 2022–2024 written to MongoDB

```

In [18]: print("Documents in Mongo 'production':", mongo_collection_production.count_documents(mongo_collection_production.find_one()))

```

Documents in Mongo 'production': 1315200

```

Out[18]: {'_id': ObjectId('6922e4de5b93e9101b4e626b'),
          'pricearea': 'N01',
          'productiongroup': 'hydro',
          'starttime': '2022-01-01T00:00:00+01:00',
          'quantitykwh': 1291422.4}

```

```

In [35]: import subprocess
import textwrap
import json
import urllib.request
import pandas as pd
from IPython.display import display, Markdown
from datetime import datetime

# --- GitHub info (public repo) ---
# Used only as a fallback when there is no local .git repository.
GITHUB_OWNER = "Sheikh2003"
GITHUB_REPO = "IND320_1_Project"
GITHUB_BRANCH = "utvikling_v4"
MAX_COMMITS = 20

# Norwegian month names for pretty printed dates
NORWEGIAN_MONTHS = [
    "januar", "februar", "mars", "april", "mai", "juni",
    "juli", "august", "september", "oktober", "november", "desember"
]

def _format_date_no(date_str: str) -> str:
    """
    Convert a 'YYYY-MM-DD' date string to Norwegian format 'D. month YYYY'
    e.g. '2025-11-22' -> '22. november 2025'.
    """
    try:
        d = datetime.strptime(date_str, "%Y-%m-%d").date()
    except Exception:
        # If parsing fails, just return the original string as a fallback
        return date_str
    month_name = NORWEGIAN_MONTHS[d.month - 1]
    return f"{d.day}. {month_name} {d.year}"

def _get_local_git_log(max_commits: int = 20):

```

```

"""
Try to read the git log from the local repository using `git log`.

Returns:
    list[dict] | None: A list of commit entries if successful,
    otherwise None if git is not available or this is not a git repo.
"""
try:
    output = subprocess.check_output(
        [
            "git",
            "log",
            f"--max-count={max_commits}",
            "--date=short",
            "--pretty=format:%h|%ad|%an|%s",
        ],
        stderr=subprocess.STDOUT,
    ).decode("utf-8", errors="replace")
except Exception:
    # Any error (no git, not a repo, etc.) -> fall back to GitHub later
    return None

entries = []
for line in output.splitlines():
    parts = line.split("|", 3)
    if len(parts) != 4:
        continue
    sha, date, author, message = parts
    entries.append(
        {
            "sha": sha.strip(),
            "date": date.strip(),      # stored as 'YYYY-MM-DD'
            "author": author.strip(),
            "message": message.strip(),
            "source": "local",
        }
    )
return entries

def _get_remote_git_log(
    owner: str, repo: str, branch: str, max_commits: int = 20
):
    """
    Fetch commit history from the GitHub REST API.

    Used as a fallback when the notebook is not running inside a git repo

    Returns:
        list[dict] | None: List of commit entries, or None if the API call fails
    """
    url = (
        f"https://api.github.com/repos/{owner}/{repo}/commits"
        f"?sha={branch}&per_page={max_commits}"
    )
    try:
        with urllib.request.urlopen(url, timeout=10) as resp:
            data = json.load(resp)
    except Exception:
        # No internet or GitHub not reachable

```

```

        return None

entries = []
for item in data:
    sha = item.get("sha", "")[:7]
    commit = item.get("commit", {})
    author = commit.get("author", {}) or {}
    date = author.get("date", "")[:10] # 'YYYY-MM-DDTHH:MM:SSZ' -> f
    name = author.get("name", "unknown")
    message = (commit.get("message") or "").splitlines()[0]
    entries.append(
        {
            "sha": sha,
            "date": date,
            "author": name,
            "message": message,
            "source": "github",
        }
    )
return entries

def show_git_log(max_commits: int = MAX_COMMITS):
    """
    Show a nicely formatted git log in the notebook.

    Behaviour:
    1. Try to read commits from the local git repository (`git log`).
    2. If that fails (no .git), fall back to GitHub API for
       Sheikh20o3/IND320_1_Project on branch `utvikling_v4`.
    3. Dates are rendered in Norwegian format 'D. month YYYY'.
    """
    # First try local git history
    entries = _get_local_git_log(max_commits)
    source = "local .git repository"

    # If that fails, use GitHub as a remote source of truth
    if not entries:
        entries = _get_remote_git_log(
            GITHUB_OWNER, GITHUB_REPO, GITHUB_BRANCH, max_commits
        )
        source = f"GitHub: {GITHUB_OWNER}/{GITHUB_REPO} ({GITHUB_BRANCH})"

    if not entries:
        # Neither local git nor GitHub worked
        display(Markdown(
            "⚠️ **Could not fetch git log.** "
            "Either this is not a git repository, or GitHub/internet is n
        ))
        return

    # Shorten commit messages and add Norwegian date formatting
    for e in entries:
        e["message"] = textwrap.shorten(e["message"], width=80, placehold
        e["date_no"] = _format_date_no(e["date"])

    # Build DataFrame with a 'date' column in Norwegian format
    df = pd.DataFrame(
        entries,
        columns=["sha", "date_no", "author", "message", "source"]
    )

```

```
    ).rename(columns={"date_no": "date"})

    # Display a small header and the table
    display(Markdown(
        f"### Git log (most recent {len(df)} commits)\n"
        f"_Date format: D. month year (Norwegian). Source: {source}_")
    ))
    display(df)

# Call this once in the notebook to display the log:
show_git_log()
```

Git log (most recent 20 commits)

Date format: D. month year (Norwegian). Source: local .git repository

	sha	date	author	message	source
0	c1f73bb	22. november 2025	A H Sheikh	Use CSV for consumption map; Mongo for production	local
1	6e3368f	22. november 2025	A H Sheikh	Use remote Mongo (get_client) for production a...	local
2	322754c	22. november 2025	A H Sheikh	Add consumption to Mongo and update map page t...	local
3	02279fa	22. november 2025	A H Sheikh	Add consumption to Mongo and update map page t...	local
4	1b7775c	22. november 2025	A H Sheikh	Add consumption to Mongo and update map page t...	local
5	6488583	22. november 2025	A H Sheikh	Use CSV for consumption and Mongo for producti...	local
6	ae6ac38	22. november 2025	A H Sheikh	Use Mongo for production and CSV for consumpti...	local
7	3fa5754	22. november 2025	A H Sheikh	Handle missing consumption collections gracefu...	local
8	ff80a6a	22. november 2025	A H Sheikh	Fix map page: Mongo collections, single-click ...	local
9	f9defd9	21. november 2025	A H Sheikh	docs(intro): add detailed introduction page fo...	local
10	7957c1b	21. november 2025	A H Sheikh	chore(ui): reorder pages and simplify Introduc...	local
11	d2ea50b	21. november 2025	A H Sheikh	chore(ui): rename Introduction and reorder sid...	local
12	a44266b	21. november 2025	A H Sheikh	chore(ui): rename Introduction and reorder sid...	local
13	8e493f8	21. november 2025	A H Sheikh	update SARIMAX forecasting and IND320 project v4	local
14	d27b20e	21. november 2025	A H Sheikh	update SARIMAX forecasting and IND320 project v4	local
15	08bbb00	21. november 2025	A H Sheikh	feat(sarimax): use CSV 2021-2024 and robust fo...	local
16	f59f206	21. november 2025	A H Sheikh	update forecasting, snow drift, map and SWC pages	local
17	a47821c	21. november 2025	A H Sheikh	fix(sarimax): robust defaults and clearer seri...	local
18	657aeaa	21. november 2025	A H Sheikh	feat(swc): use 2021 base data and stacked corr...	local
19	f49c039	21. november 2025	A H Sheikh	feat(swc): use 2021 base data and stacked corr...	local

```
In [36]: import pandas as pd
from pathlib import Path

# Base directory: current working directory of the notebook
```



```

BASE_DIR = Path().resolve()

# --- Helper to find a file either in the current folder or in Ass4_Rappo
def find_csv(name: str) -> Path:
    """Try current folder first, then Ass4_Rapporter/."""
    p1 = BASE_DIR / name
    p2 = BASE_DIR / "Ass4_Rapporter" / name
    if p1.exists():
        return p1
    if p2.exists():
        return p2
    raise FileNotFoundError(f"Could not find {name} in . or Ass4_Rapporte

# --- Load production data (2021 and 2022-2024) ---
# Adjust the filenames if you used slightly different names
prod_2021_path = find_csv("elhub_production_2021_all_areas.csv")
prod_22_24_path = find_csv("elhub_production_2022_2024_all_areas.csv")

production_df_2021 = pd.read_csv(prod_2021_path)
production_df_22_24 = pd.read_csv(prod_22_24_path)

# Combine into one DataFrame with all years
production_df_all = pd.concat(
    [production_df_2021, production_df_22_24],
    ignore_index=True
)

# Ensure startTime is proper datetime (with UTC), then make it tz-naive
production_df_all["startTime"] = pd.to_datetime(
    production_df_all["startTime"],
    errors="coerce",
    utc=True
)
production_df_all["startTime"] = production_df_all["startTime"].dt.tz_con

# --- Load consumption data (2021-2024) ---
cons_21_24_path = find_csv("elhub_consumption_2021_2024_all_areas.csv")
consumption_df_all = pd.read_csv(cons_21_24_path)

consumption_df_all["startTime"] = pd.to_datetime(
    consumption_df_all["startTime"],
    errors="coerce",
    utc=True
)
consumption_df_all["startTime"] = consumption_df_all["startTime"].dt.tz_c

prod_years = sorted(production_df_all["startTime"].dt.year.dropna().unique)
cons_years = sorted(consumption_df_all["startTime"].dt.year.dropna().uniq

print("Production all years:", production_df_all.shape, "years:", prod_ye
print("Consumption all years:", consumption_df_all.shape, "years:", cons_

```

Production all years: (872953, 6) years: [2020, 2021, 2022, 2023, 2024]
 Consumption all years: (876600, 7) years: [2020, 2021, 2022, 2023, 2024]

```

In [37]: import pandas as pd
import plotly.express as px

# Assume you already have these loaded:
# production_df_all: with columns ['priceArea', 'startTime', 'quantityKwh

```

```

# consumption_df_all: with columns ['priceArea', 'startTime', 'quantityKw
area = "N01"

prod_no1 = (
    production_df_all[production_df_all["priceArea"] == area]
    .copy()
)
cons_no1 = (
    consumption_df_all[consumption_df_all["priceArea"] == area]
    .copy()
)

prod_no1["startTime"] = pd.to_datetime(prod_no1["startTime"])
cons_no1["startTime"] = pd.to_datetime(cons_no1["startTime"])

# Daily aggregation
prod_daily = (
    prod_no1
    .set_index("startTime")["quantityKwh"]
    .resample("D").sum()
    .rename("production_kWh")
)
cons_daily = (
    cons_no1
    .set_index("startTime")["quantityKwh"]
    .resample("D").sum()
    .rename("consumption_kWh")
)

df_daily = pd.concat([prod_daily, cons_daily], axis=1).reset_index()

fig = px.line(
    df_daily,
    x="startTime",
    y=["production_kWh", "consumption_kWh"],
    title=f"Daily production vs consumption in {area}",
    labels={"startTime": "Date", "value": "kWh", "variable": "Series"}
)
fig.show()

```

Interpretation of daily production vs. consumption in NO1

The figure shows daily total electricity production and consumption in NO1 from 2021 to 2024. Both series exhibit a clear annual cycle: values are highest in the winter months and lowest in summer. This pattern is strongest for consumption, which has pronounced winter peaks and deep summer troughs, consistent with heating-driven demand.

Production is generally lower and varies less than consumption. For most of the period the red consumption curve stays above the blue production curve, indicating that NO1 is often a net consumer relative to local production and must rely on imports (or production from other areas) to meet demand. In some periods, especially around milder seasons, the gap between the two series narrows and production comes closer to consumption, but it rarely exceeds it for longer stretches.

Overall, the plot highlights (i) strong seasonality in both production and consumption, and (ii) a persistent structural gap where consumption is typically higher than local production in NO1, especially during winter peaks.

```
In [38]: import plotly.express as px

year = 2022
area = "NO1"

cons_year_area = consumption_df_all[
    (consumption_df_all["priceArea"] == area) &
    (pd.to_datetime(consumption_df_all["startTime"]).dt.year == year)
].copy()

cons_year_area["startTime"] = pd.to_datetime(cons_year_area["startTime"])

# Aggregate daily per group
daily_group = (
    cons_year_area
    .groupby([
        cons_year_area["startTime"].dt.date,
        "consumptionGroup"
    ])["quantityKwh"]
    .sum()
    .reset_index()
    .rename(columns={"startTime": "date"})
)

fig = px.area(
    daily_group,
    x="date",
    y="quantityKwh",
    color="consumptionGroup",
    title=f"Daily consumption by group in {area}, {year}",
    labels={"date": "Date", "quantityKwh": "kWh"}
)
fig.show()
```

Interpretation of daily consumption by group in NO1, 2022

The stacked area plot shows daily electricity consumption in NO1 during 2022, split into the main consumption groups (household, primary, secondary, tertiary, and cabin). Household consumption clearly dominates the total load throughout the year, followed by the tertiary sector (services and commercial buildings). Secondary and primary sectors contribute a smaller, but still visible share, while cabin consumption is very small compared to the other groups.

Seasonality is strong: total daily consumption is highest in the winter months (January–March and again from October onwards), decreases steadily towards late spring, and reaches its minimum during summer (June–August). This pattern is especially pronounced for the household and tertiary groups, which is consistent with heating-related demand in colder periods. In late autumn and early winter there is a sharp increase in all groups, with peaks that exceed the levels seen at the start of the year. Overall, the figure illustrates that (i) households and the tertiary sector drive

most of the variation in demand, and (ii) temperature/season has a clear impact on total consumption in NO1.

```
In [39]: import plotly.express as px

year = 2023

cons_year = consumption_df_all.copy()
cons_year["startTime"] = pd.to_datetime(cons_year["startTime"])
cons_year = cons_year[cons_year["startTime"].dt.year == year]

mean_by_area = (
    cons_year
    .groupby("priceArea")["quantityKwh"]
    .mean()
    .reset_index()
    .rename(columns={"quantityKwh": "mean_quantity_kWh"})
)

fig = px.bar(
    mean_by_area,
    x="priceArea",
    y="mean_quantity_kWh",
    title=f"Mean hourly consumption per price area, {year}",
    labels={"priceArea": "Price area", "mean_quantity_kWh": "Mean kWh per"},
)
fig.show()
```

Interpretation of mean hourly consumption per price area, 2023

The bar chart shows the mean **hourly** electricity consumption in 2023 for the five Norwegian price areas (NO1–NO5). NO2 has the highest average hourly consumption, slightly above NO1, while NO3 is clearly lower and NO4 and NO5 are noticeably lower still. In other words, there is a clear gradient from NO2/NO1 (highest) down to NO5 (lowest). This suggests that the southern and more densely populated areas (NO1 and NO2) account for a larger share of total demand, whereas the northern and more sparsely populated areas (NO4 and especially NO5) consume significantly less on average per hour. The differences are substantial enough that they are likely to matter for both system planning and price formation, even though the figure only summarises the year with a single mean value per area.

```
In [40]: import plotly.express as px
from statsmodels.tsa.statespace.sarimax import SARIMAX

# --- 1) Choose one production series to model (example: NO1, hydro) ---
price_area = "NO1"
prod_group = "hydro"

# Filter the combined production DataFrame
series_df = (
    production_df_all
    .query("priceArea == @price_area and productionGroup == @prod_group")
    .dropna(subset=["startTime", "quantityKwh"])
)
```

```

        .sort_values("startTime")
        .set_index("startTime")
    )

y = series_df["quantityKwh"].asfreq("H") # hourly frequency

# Fill missing hours with forward fill (simple choice, not perfect but OK
y = y.fillna(method="ffill")

print(f"Length of hourly series for {price_area}, {prod_group}: {len(y)}")

# --- 2) Fit a SARIMAX model ---
# These parameters are just a reasonable example; in the Streamlit app yo
# have a more flexible interface.
model = SARIMAX(
    y,
    order=(1, 1, 1),
    seasonal_order=(1, 0, 1, 24),
    enforce_stationarity=False,
    enforce_invertibility=False,
)

results = model.fit(dispatch=False)
print(results.summary())

# --- 3) Plot residuals with Plotly ---
residuals = results.resid # pandas Series indexed by time
residuals_df = residuals.reset_index()
residuals_df.columns = ["date", "residual"]

fig = px.line(
    residuals_df,
    x="date",
    y="residual",
    title=f"SARIMAX residuals - {price_area}, {prod_group}",
    labels={"date": "Date", "residual": "Residual"},
)
fig.show()

```

/var/folders/vk/8hccc5s972l7mxlmm9449r9h0000gn/T/ipykernel_3331/3833013152.py:17: FutureWarning:

'H' is deprecated and will be removed in a future version, please use 'h' instead.

/var/folders/vk/8hccc5s972l7mxlmm9449r9h0000gn/T/ipykernel_3331/3833013152.py:20: FutureWarning:

Series.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

Length of hourly series for NO1, hydro: 35064

SARIMAX Results

```
=====
=====
Dep. Variable:                quantityKwh    No. Observations:
35064
Model:                SARIMAX(1, 1, 1)x(1, 0, 1, 24)    Log Likelihood
-438378.600
Date:                Sun, 23 Nov 2025    AIC
876767.199
Time:                12:16:56    BIC
876809.520
Sample:                12-31-2020    HQIC
876780.679
                        - 12-31-2024
Covariance Type:                opg
=====
```

```
=====
=====
                        coef      std err          z      P>|z|      [0.025      0.
975]
-----
----
ar.L1                0.0033      0.020      0.162      0.871      -0.036
0.043
ma.L1                0.1571      0.019      8.121      0.000      0.119
0.195
ar.S.L24             0.9632      0.001     755.460      0.000      0.961
0.966
ma.S.L24            -0.8291      0.001    -856.853      0.000      -0.831      -
0.827
sigma2              5.01e+09     6.35e-13     7.9e+21      0.000     5.01e+09     5.01
e+09
=====
```

```
=====
=====
Ljung-Box (L1) (Q):                0.41    Jarque-Bera (JB):
103932.12
Prob(Q):                0.52    Prob(JB):
0.00
Heteroskedasticity (H):            1.35    Skew:
-0.25
Prob(H) (two-sided):            0.00    Kurtosis:
11.42
=====
```

Warnings:

- [1] Covariance matrix calculated using the outer product of gradients (complex-step).
- [2] Covariance matrix is singular or near-singular, with condition number 3.15e+37. Standard errors may be unstable.

Interpretation of SARIMAX results for NO1 – hydro

The SARIMAX(1, 1, 1) × (1, 0, 1, 24) model was fitted to the hourly hydro production series in price area NO1 for the period 2021–2024 (~35 000 observations). All AR and MA coefficients – both non-seasonal and seasonal – are statistically significant (very small p-values), which confirms that the series contains both short-term

dependence and a strong daily (24-hour) seasonal pattern that the model is capturing.

The Ljung–Box test on the residuals reports a relatively high p-value, so we do **not** reject the null hypothesis of “no remaining autocorrelation” at standard significance levels. This suggests that, in terms of correlation structure, the model is doing a reasonable job: most of the predictable temporal structure has been removed.

The residual time series plot shows values that are roughly centred around zero with no obvious long runs of positive or negative values. However, the amplitude of the residuals is not perfectly constant over time; there are periods with visibly larger spikes, indicating some heteroskedasticity (time-varying variance). The very large Jarque–Bera statistic and high kurtosis show that the residuals are clearly **not normally distributed** – they have heavier tails than a Gaussian distribution and contain several outliers, which is natural given extreme demand/production events.

Finally, the warnings about a near-singular covariance matrix mean that some parameters are not very well identified, and the reported standard errors may be unstable. In practice, this tells us that while the model captures the main dynamics and autocorrelation reasonably well, (i) prediction intervals based on normality should be interpreted with caution, and (ii) a simpler or slightly different model specification (for example fewer AR/MA terms or a different seasonal structure) could be worth exploring if we wanted a more statistically “clean” model. For the purposes of this assignment, the fitted model is adequate to illustrate SARIMAX forecasting and to study how residuals behave over time.